

EIGER QSO GUI: User Guide

Interactive Quasar Spectroscopy and Voigt Profile Fitting

EIGER Collaboration

June 2026

Contents

1	Introduction	3
1.1	Environment Variables	3
1.2	Python Dependencies	3
2	Launching the GUI	4
3	Loading and Navigating a Spectrum	5
3.1	Selecting an Object	5
3.2	Choosing Which Instruments to Display	5
3.3	Keyboard Navigation	5
4	Identifying Absorption Lines	7
4.1	Setting a Trial Redshift	7
4.2	Setting Redshift by Line Identification	7
4.3	Quick Doublet Checks	7
4.4	Building the ID Table	7
5	Registering a Local Object	9
5.1	The <code>local_objects.yaml</code> File	9
5.2	Instrument Keys	9
5.3	Continuum Modes	9
5.4	Verifying Registration	10
6	Building a Voigt Profile Model	11
6.1	Opening the VP Model Tree	11
6.2	The Four-Level Hierarchy	11
6.3	Adding Components, Ions, and Transitions Interactively	11
6.4	Right-Click Context Menu	12
6.5	Defining Fit Regions	12
6.6	The Live Model Preview	12
6.7	Saving and Loading Model Files	13
7	Running the Voigt Profile Fit	14

7.1	The Model (.in) File Format	14
7.2	Fitting a Built-in EIGER Object	14
7.3	Fitting a Locally-Registered Object	15
7.4	Instrumental Resolution Kernels	15
7.5	What to Expect During Execution	15
8	Evaluating Fit Results	17
8.1	Loading a Pickle File	17
8.2	Printed Parameter Table	17
8.3	Assessing Fit Quality	17
8.4	Loading Pre-computed FITS Profile Files	18
9	Keyboard Shortcut Reference	19
9.1	Navigation	19
9.2	Line Identification	19
9.3	Model Building (requires ID table to be populated)	19
9.4	File I/O	19
10	Quick-Start Checklist	20

1 Introduction

The EIGER QSO GUI is an interactive PyQt5 application for inspecting high-redshift quasar absorption spectra, building multi-component Voigt profile models, and evaluating fits produced by the MCMC sampler. The intended workflow is:

1. **Load** a quasar spectrum from the EIGER data archive or from local disk.
2. **Identify** absorption line systems by setting a trial redshift and labelling features on the plot.
3. **Model** the absorber by constructing a hierarchical component tree that specifies the velocity components, ions, transitions, and fit regions.
4. **Export** the model to a `.in` file and run the MCMC Voigt profile fitter (`vpfit_template.py` or a sightline-specific `vpfit.py`) from the command line.
5. **Evaluate** the fit by loading the output pickle file back into the GUI, which overlays the posterior profile ensemble on the spectrum.

This guide assumes familiarity with quasar absorption spectroscopy but no prior experience with the GUI.

1.1 Environment Variables

The GUI depends on two shell environment variables that must be set before launching:

Variable	Purpose
EIGER_PATH	Root of the local <code>eiger</code> package tree. Used to locate atomic line data (<code>atomic_data.txt</code>).
EIGER_CACHE	Directory that holds locally-cached spectra and the <code>local_objects.yaml</code> registry.

Add lines such as the following to your `.bash_profile` or equivalent:

```
export EIGER_PATH=/path/to/eiger
export EIGER_CACHE=/path/to/eiger_cache
```

1.2 Python Dependencies

The GUI requires the following packages in addition to the base `eiger` library:

- PyQt5
- matplotlib (Qt5Agg backend)
- astropy
- numpy
- mcvp (the EIGER Voigt profile modelling library)
- pypeit (for air-to-vacuum wavelength correction on HIRES data)

2 Launching the GUI

Navigate to the `eigerQsoGUI` directory and run:

```
python eigerqso.py
```

The main window opens with an empty plot panel, a toolbar, and a set of controls on the right-hand side. The spectrum display area will be blank until an object is loaded.

[FIGURE 1: Main window on startup — empty plot area with controls visible on the right: object dropdown, Reload button, Select Spectra button, VP Tree button, Load VPfit button, and the redshift entry field.]

3 Loading and Navigating a Spectrum

3.1 Selecting an Object

The dropdown at the top-right of the window lists all available objects. The first six entries are the standard EIGER quasars:

Index	Object
1	J1030+1524
2	P159-02
3	J1120+0641
4	J0100+2802
5	J1148+5251
6	J0148+0600

Objects registered in `local_objects.yaml` (see Section 5) appear below these as items 7, 8, ...

Select the desired object from the dropdown and press **Reload**. The GUI loads the spectrum from the EIGER archive (or from local disk for custom objects), normalises each instrument arm by its continuum, and displays the first three available instrument arms as separate sub-panels stacked vertically with a shared wavelength axis. The initial wavelength range is 8000–23000 Å.

[FIGURE 2: Main window after loading a quasar, showing three instrument panels (e.g. XSH_NIR, HIRES, XSH_VIS) with flux/continuum on the y-axis, wavelength in Å on the x-axis, and the red 1σ error spectrum plotted at low opacity.]

3.2 Choosing Which Instruments to Display

The GUI always plots exactly three panels. To change which instrument arms are shown, or to toggle a panel on or off, press **Select Spectra**. A dialog appears with three dropdown menus — one per panel — listing all available arms for the loaded object. Checkboxes next to panels 2 and 3 allow those panels to be hidden entirely if fewer than three arms are needed.

3.3 Keyboard Navigation

With the cursor inside the plot area, the following single-key shortcuts control the viewport. Note that the canvas must have keyboard focus; click once on the plot area first if shortcuts are not responding.

Key	Action
l	Set left edge of wavelength window to cursor position
r	Set right edge of wavelength window to cursor position
t	Set top of flux window to cursor position
b	Set bottom of flux window to cursor position
w / W	Zoom to full wavelength range of all loaded arms
] or }	Pan right by one full window width

Key	Action
[or {	Pan left by one full window width
o	Zoom out by 50% around the window centre
i	Zoom in by ~40%, recentred on the cursor

The standard matplotlib navigation toolbar (zoom box, pan, home, back, forward) at the bottom of the plot window is also available for mouse-driven navigation.

4 Identifying Absorption Lines

Before building a Voigt profile model, you will typically want to identify the absorption features present in the spectrum and assign them redshifts. The GUI provides several complementary tools for this.

4.1 Setting a Trial Redshift

Type a redshift (e.g. 5.1080) into the `z =` text field at the bottom-right of the window and press Return. The GUI will redshift all lines in the atomic line list (`atomic_data.txt`) and draw thin vertical markers at each predicted observed wavelength that falls within the current view. Line labels (ion name and rest wavelength) are printed in small text at the top of each marker.

Changing the viewport and pressing any navigation key will refresh the labels for the new wavelength window.

[FIGURE 3: Spectrum with trial redshift set, showing vertical line markers with ion labels (e.g. CIV 1548, CIV 1550, MgII 2796) overlaid on the absorber region.]

4.2 Setting Redshift by Line Identification

Right-clicking anywhere on the spectrum opens the line identification dialog, which lists the full atomic line list with ion name and rest wavelength. Select the line you are identifying, then click OK. The GUI computes the redshift implied by the observed wavelength of the click divided by the selected rest wavelength, sets the redshift field accordingly, and redraws the line markers.

4.3 Quick Doublet Checks

Three keyboard shortcuts draw bracket markers that show where the second member of a common doublet should fall given the first:

Key	Doublet
C	C IV 1548, 1550 doublet
M	Mg II 2796, 2803 doublet
S	C II 1334 / C II* 1335

Place the cursor on the blue member of the doublet and press the key. A red bracket is drawn connecting the two predicted line positions. This is useful for confirming candidate identifications before committing a redshift.

4.4 Building the ID Table

Clicking on any line label in the plot (the text items placed by the redshift labelling routine) adds that line to an in-memory ID table, which stores the ion name, rest wavelength, and redshift for each identified feature. The table is printed to the terminal after each addition.

- Press **F** to save the current ID table to an ASCII fixed-width file (a file dialog appears).

- Press **R** to read a previously saved ID table back in. Saved entries are drawn in green on the spectrum to distinguish them from the current trial-redshift labels.

5 Registering a Local Object

The six built-in EIGER targets load spectra automatically from the AWS archive. To use the GUI with your own data you must first register the object in a YAML file on local disk.

5.1 The `local_objects.yaml` File

Create or edit `$EIGER_CACHE/local_objects.yaml`. A template with comments is provided in `eigerQsoGUI/local_objects_example.yaml`. The file has the structure:

`objects:`

```
- name: "J0000+0000"      # Name as it will appear in the GUI dropdown
  z_em: 6.10              # Emission redshift (informational)
```

`spectra:`

`FIRE:`

```
- path: "Magellan/FIRE/J0000+0000_FIRE.fits"
  continuum: companion    # see continuum modes below
```

`XShooter:`

```
- path: "VLT/XShooter/J0000+0000_VIS.fits"
  continuum: companion
  continuum_path: "VLT/XShooter/J0000+0000_VIS_contin.fits"
- path: "VLT/XShooter/J0000+0000_NIR.fits"
  continuum: companion
  continuum_path: "VLT/XShooter/J0000+0000_NIR_contin.fits"
```

All paths are relative to `$EIGER_CACHE`.

5.2 Instrument Keys

The instrument key used under `spectra:` determines how the arm is labelled in the GUI. Recognised keys and their GUI labels are:

YAML key	GUI / <code>vpfit.py</code> label
FIRE	FIRE
XShooter (two files: VIS then NIR)	XSH_VIS, XSH_NIR
HIRES	HIRES
MOSFIRE_Y / _J / _H / _K	MOSFIRE_Y ... MOSFIRE_K
NIRSpec	NIRSpec

5.3 Continuum Modes

Each spectrum file entry requires a `continuum` field specifying how the normalisation is provided:

Mode	Description
<code>normalized</code>	The FITS file already contains flux divided by the continuum; no separate continuum file is needed.
<code>companion</code>	The continuum is in a separate FITS file. By default the loader looks for <code><spectrum_name>_contin.fits</code> alongside the spectrum. Override with an explicit <code>continuum_path</code> .
<code>extension</code>	The continuum is in a different HDU of the same FITS file. Specify <code>continuum_hdu</code> (integer, default 2) and <code>continuum_column</code> (column name, default "cont").

5.4 Verifying Registration

After saving `local_objects.yaml`, restart the GUI. The new object should appear at the bottom of the dropdown list (items 7 and above). Select it and press Reload to verify that the spectra load correctly.

6 Building a Voigt Profile Model

6.1 Opening the VP Model Tree

Press **VP Tree** in the main window. A separate dialog opens containing a hierarchical tree widget and three buttons: **Write**, **Read**, and **Clear**.

[**FIGURE 4: VP Model Tree window, showing the four-level hierarchy (Component at level 0, Ion at level 1, Transition at level 2, Fit Region at level 3) with one or two example entries populated. The column headers “Component”, “b (km/s)”, and “N (cm-2)” are visible.**]

The tree has three columns:

Column	Level 0 (Component)	Level 1 (Ion)	Level 2 (Transition)	Level 3 (Fit Region)
Component	Absorption redshift z	Ion name	Rest wavelength (Å)	Instrument + velocity range
b (km/s)	Doppler b parameter	—	—	—
log N (cm-2)	—	log10 N	—	—

The b-parameter and log N cells are directly editable: double-click to change a value. The cyan model profile in the main spectrum window updates live after each edit.

6.2 The Four-Level Hierarchy

Each absorbing system is represented as a tree with four levels:

Component (level 0) — a kinematic component defined by a redshift and a Doppler b parameter. Multiple components at different redshifts model a velocity-spread absorber.

Ion (level 1) — an ionic species (e.g. C IV, Mg II) that is detected in a given component. Each ion carries a log column density.

Transition (level 2) — a specific rest-wavelength transition of the parent ion (e.g. C IV 1548 Å). Multiple transitions of the same ion can be included to use all available wavelength coverage as simultaneous constraints.

Fit Region (level 3) — a velocity interval [$v_{\min}$, $v_{\max}$] km/s (relative to the parent transition’s expected wavelength) within which the MCMC sampler will compare the model to the data. Each transition should have at least one fit region.

6.3 Adding Components, Ions, and Transitions Interactively

The most efficient way to build a model is through keyboard shortcuts on the main spectrum plot. These shortcuts act on the nearest **identified line** — the ID table must contain at least one entry before they can be used (see Section 4.4).

Adding a component (c): Place the cursor at the centre of an absorption feature and press c. The GUI finds the closest entry in the ID table by observed wavelength, computes the implied

redshift, and adds a new component at that redshift to the tree. A confirmation message is printed to the terminal.

Adding an ion (I): Place the cursor on a feature and press I. The GUI identifies the nearest ID table line and adds that ion to the component whose redshift is closest to the implied value. If no component within $\Delta z = 0.005$ exists, a warning is printed.

Adding a transition (T): Place the cursor on a feature and press T. The GUI identifies the nearest ID table entry and appends that rest-wavelength transition to the appropriate ion in the tree.

6.4 Right-Click Context Menu

Right-clicking on a selected tree row opens a context menu whose contents depend on the level:

- **Component (level 0):** Lists all ion names from the atomic line list. Selecting an ion adds it as a child of the selected component. A “Delete Component” option is also present.
- **Ion (level 1):** Lists all rest-wavelength transitions available for that ion. Selecting a wavelength adds a new transition child. A “Delete Ion” option is present.
- **Transition (level 2):** “Delete Transition” only.
- **Fit Region (level 3):** “Delete FitRegion” only.

6.5 Defining Fit Regions

Fit regions are defined graphically by Shift+click-dragging on any instrument panel in the main spectrum window.

1. In the VP Model Tree, **click to select the transition** (level 2 row) to which you want to attach the region.
2. In the main spectrum, hold **Shift** and click-drag from the left to the right edge of the pixel range you want to include in the fit. Release the mouse button.

The GUI converts the dragged wavelength range to a velocity interval relative to the selected transition’s expected wavelength at its parent component’s redshift, and appends a new Fit Region child to the selected transition. The instrument arm is determined automatically from which panel the drag occurred in.

[FIGURE 5: Main spectrum window with a shaded or bracketed fit region visible on one of the instrument panels, and the corresponding Fit Region entry highlighted in the tree widget.]

6.6 The Live Model Preview

As components, ions, and transitions are added or edited, the GUI computes and displays a cyan model profile on the main spectrum. This uses the current parameter values from the tree (redshift, b , $\log N$) convolved with the appropriate instrumental resolution kernel. The preview lets you judge whether your initial guesses are reasonable before committing to a multi-hour MCMC run.

[FIGURE 6: Spectrum with the cyan initial-guess Voigt profile overlaid on an absorber, showing reasonable alignment with the observed trough depths.]

6.7 Saving and Loading Model Files

Write (in the VP Tree dialog) opens a save dialog and writes the current tree contents to a plain-text `.in` file (see Section 7.1 for the format). This file is the direct input to the fitting scripts.

Read opens a load dialog and populates the tree from a previously saved `.in` file. This is useful for resuming work on a partially built model or for loading a model from a previous analysis run.

Clear removes all entries from the tree and resets the internal model object.

7 Running the Voigt Profile Fit

7.1 The Model (.in) File Format

The `.in` file is a plain-text list of Python method calls that construct the model object `m`. A minimal example for a two-component CIV absorber looks like:

```
m.addcomponent(5.1075, bpriors=[3,50], b_turb=10.0)
m.addion('CIV', 5.1075, N=12.0, Npriors=[11,15])
m.addtransition(1548.0, 'CIV', 5.1075)
m.addfitregion(1548.0, 'CIV', 5.1075, [-20.0, 10.0], 'HIRES')
m.addtransition(1550.0, 'CIV', 5.1075)
m.addfitregion(1550.0, 'CIV', 5.1075, [-22.0, 20.0], 'HIRES')

m.addcomponent(5.1108, bpriors=[3,50], b_turb=10.0)
m.addion('CIV', 5.1108, N=12.0, Npriors=[11,15])
m.addtransition(1548.0, 'CIV', 5.1108)
m.addfitregion(1548.0, 'CIV', 5.1108, [-16.0, 12.0], 'HIRES')
m.addtransition(1550.0, 'CIV', 5.1108)
m.addfitregion(1550.0, 'CIV', 5.1108, [-19.0, 20.0], 'HIRES')
```

The four call types and their arguments are:

```
m.addcomponent(z, bpriors=[lo,hi], b_turb=b0)
```

Adds a velocity component at redshift `z`. `bpriors` sets the uniform prior range on the Doppler `b` parameter in km/s. `b_turb` is the initial guess.

```
m.addion(ion, z, N=N0, Npriors=[lo,hi])
```

Adds ionic species `ion` (string, e.g. 'CIV', 'MgII') to the component at redshift `z`. `N` is the initial log10 column density guess and `Npriors` sets the uniform prior range in log10 units.

```
m.addtransition(restwv, ion, z)
```

Registers rest wavelength `restwv` (Å) as an observed transition for ion `ion` at component redshift `z`. Multiple transitions can be added for the same ion to leverage all available spectral coverage.

```
m.addfitregion(restwv, ion, z, [vmin, vmax], instrument)
```

Defines a fit region for transition `restwv` of ion `ion` at component `z`. The region is specified as a velocity interval `[vmin, vmax]` in km/s relative to the line centre, on instrument arm `instrument`.

7.2 Fitting a Built-in EIGER Object

Each of the six standard EIGER sightlines has a dedicated fitting script in its subdirectory (J1030/vpfit.py, P159/vpfit.py, etc.). These scripts hard-code the correct object index and instrument arms. Run from within the sightline directory:

```
cd J1030
python vpfit.py mymodel.in output.pickle
```

7.3 Fitting a Locally-Registered Object

For objects registered in `local_objects.yaml`, use the generic template:

```
python vpfit_template.py mymodel.in output.pickle
```

Before running, open `vpfit_template.py` and set two configuration variables near the top of the file:

```
OBJ_NAME      = "J0000+0000"          # must match name in local_objects.yaml
INSTRUMENTS = ['FIRE', 'XSH_VIS', 'XSH_NIR'] # arms to include in fit
```

Include only the arms for which the absorber's transitions fall within the spectral coverage.

7.4 Instrumental Resolution Kernels

The fitting scripts convolve the Voigt profile with a Gaussian instrumental line-spread function. The default kernel widths are:

Instrument	Slit / Config	FWHM (pixels)	Gaussian sigma (pixels)
FIRE	0.6" slit, 0.15"/pix	4.0	1.70
XSH_NIR	0.9" slit	2.2	0.93
XSH_VIS	0.9" slit	4.8	2.04
HIRES	0.86" slit (6 km/s)	3.0	1.27
MOSFIRE Y	0.7" slit, R=3380	~2.8	1.19
MOSFIRE J	0.7" slit, R=3310	~2.9	1.23
MOSFIRE H	0.7" slit, R=3660	~2.8	1.17
MOSFIRE K	0.7" slit, R=3620	~2.8	1.18
NIRSpec	G140H/G235H	2.2	0.93

If you used a non-standard slit width or a different instrument configuration, adjust the `kernels` dictionary in the template script before running.

7.5 What to Expect During Execution

The script prints progress messages to the terminal:

```
Loading J0000+0000 spectra...
  Added FIRE
  Added XSH_VIS
  Added XSH_NIR
Ncomponents=2, Nions=2, Nparams=6
Running the sampler...
```

The MCMC sampler runs with `nwalkers = 2 × Nparams` walkers for 3000 steps. For a typical 5–10 component system with 2–3 ions this takes roughly 15–60 minutes on a modern laptop. The number of free parameters is:

$$N_{\text{params}} = 2 \times N_{\text{components}} + N_{\text{ions}}$$

where each component contributes a redshift and a b parameter, and each ion (per component) contributes a log column density.

On completion, the sampler writes two outputs:

- **output.pickle** — a Python pickle containing the model object and the MCMC chain. This is what you load back into the GUI.
- **vpfit_corner.pdf** — a corner plot of the posterior samples for quick visual inspection of parameter covariances (produced by the sightline-specific scripts).

8 Evaluating Fit Results

8.1 Loading a Pickle File

Press **Load VPfit** in the main window. A file dialog opens; navigate to and select the output pickle file. The GUI will:

1. Reconstruct the model from the stored `mcvp` model object.
2. Draw 50 random samples from the posterior chain (discarding the first 1000 steps as burn-in).
3. Evaluate and convolve the Voigt profile for each sample on each loaded instrument arm.
4. Plot each of the 50 profile realisations in red at low opacity ($\alpha = 0.2$) on top of the data.

The resulting ensemble of red profiles gives a direct visual impression of both the best-fit model shape and its posterior uncertainty.

[FIGURE 7: Spectrum showing the red posterior profile ensemble (50 semi-transparent draws) overlaid on the data, with the data plotted in the default stepped histogram style and the 1σ error array in light red. The profiles bracket the absorption feature.]

8.2 Printed Parameter Table

After loading, the GUI prints a formatted parameter table to the terminal. Each row contains the ion name and the 50th, 16th, and 84th percentile of the marginal posterior for redshift, b parameter, and log column density:

ion	z_median	z_16pct	z_84pct	b_median	...	N_median	N_16pct	N_84pct
CIV	5.10750	5.10743	5.10757	8.2	...	13.45	13.38	13.52
CIV	5.11080	5.11071	5.11089	11.4	...	12.83	12.71	12.95
...								

The table is sorted in order of decreasing redshift (blue-to-red in velocity space).

8.3 Assessing Fit Quality

When reviewing the result, check the following:

- **Profile alignment:** The red ensemble should be centred on the absorption troughs. A systematic offset suggests the initial redshift guess was too far from the true value for the sampler to converge.
- **Profile spread:** A narrow red bundle indicates well-constrained parameters. A wide spread may indicate a poorly constrained transition (e.g. saturated absorption) or degeneracies between components that could be broken by adding additional transitions.
- **Residuals:** The data should scatter uniformly around the model profiles within the fit regions. Systematic residuals (data consistently above or below all 50 profiles) indicate that the number of components or the chosen velocity range is inadequate.
- **Parameter posteriors:** Review `vpfit_corner.pdf` to check that the posterior distributions are unimodal and well-sampled. Bimodal or rail-hitting posteriors indicate modelling

issues that should be addressed before reporting results.

8.4 Loading Pre-computed FITS Profile Files

For cases where the 50-sample ensemble has been pre-computed and stored in FITS format (e.g. the DR1 release files in `EIGER_absline_results/dr1/`), the **Load VPfit** button can also accept these files. The FITS file must contain columns named `modelprof0` through `modelprof49`, and the filename must include the instrument arm name (e.g. `..._xsh_nir_modelprofs...`, `..._hires_modelprofs...`, `..._fire_modelprofs...`, etc.) so that the GUI can route the profiles to the correct panel.

You can load profiles for multiple arms sequentially by pressing **Load VPfit** multiple times — each call adds profiles for one arm without clearing the others.

[FIGURE 8: Same spectrum as Figure 7, but with profiles loaded for two arms (e.g. HIRES and XSH_NIR) simultaneously, showing the red ensemble on both panels.]

9 Keyboard Shortcut Reference

9.1 Navigation

Key	Action
l	Set left wavelength bound to cursor
r	Set right wavelength bound to cursor
t	Set upper flux limit to cursor
b	Set lower flux limit to cursor
w / W	Full wavelength zoom (all arms)
] or }	Pan right one window width
[or {	Pan left one window width
o	Zoom out 50% around centre
i	Zoom in ~40%, centred on cursor

9.2 Line Identification

Key	Action
Right-click	Open line ID dialog at cursor wavelength
C	Draw C IV 1548/1550 doublet bracket at cursor
M	Draw Mg II 2796/2803 doublet bracket at cursor
S	Draw C II 1334 / C II* 1335 bracket at cursor
F	Save ID table to ASCII file (file dialog)
R	Read ID table from ASCII file (file dialog)

9.3 Model Building (requires ID table to be populated)

Key	Action
c	Add velocity component at cursor (z from nearest ID line)
I	Add ion to nearest component (ion from nearest ID line)
T	Add transition to nearest ion (rest wavelength from nearest ID line)
Shift+drag	Define fit region on selected transition

9.4 File I/O

Key	Action
F	Save ID table (see above)
R	Read ID table (see above)
VP Tree -> Write	Save model to .in file
VP Tree -> Read	Load model from .in file
Load VPfit button	Load MCMC pickle or FITS profiles

10 Quick-Start Checklist

The following checklist summarises the end-to-end process for a new absorber:

- ☐ Set `EIGER_PATH` and `EIGER_CACHE` environment variables
- ☐ If using a local object, add an entry to `$EIGER_CACHE/local_objects.yaml`
- ☐ Launch `python eigerqso.py`
- ☐ Select the target from the dropdown; press **Reload**
- ☐ Navigate to the absorber wavelength region; zoom with `l`, `r`, `t`, `b`
- ☐ Set a trial redshift in the `z` field and press Return; verify line labels
- ☐ Right-click features to confirm identifications; build the ID table
- ☐ Press **VP Tree** to open the model tree
- ☐ Press `c` at each velocity component to add components to the tree
- ☐ Press `I` and `T` as needed to add ions and transitions
- ☐ Shift+drag on each transition to define fit regions
- ☐ Inspect the cyan guess profile; edit `b` and `N` values in the tree if needed
- ☐ Press **Write** to save the model to a `.in` file
- ☐ Run `python vpfit_template.py mymodel.in output.pickle` (or the sightline-specific script)
- ☐ After the sampler finishes, press **Load VPfit** and select `output.pickle`
- ☐ Review the red posterior ensemble and the printed parameter table
- ☐ Check `vpfit_corner.pdf` for posterior convergence